Step 0

Step 0-1 → read .html file (before game.js).

- Resonance Audio : ResonanceAudio,
- air-absorption.js : install Air Absorption.
- front-back.js : install FrontBack.
- minimap.js : drawMinimap.
- game.js : my own code. load after ↑ all those things.

Step 0-2 → read game.js.

```
let ctx, scene, voices = [], statusBox;
let keys = {};
let enemyAudioBuffers = [];
let player = {x: 0, y: -2, heading: Math.PI/2};
const speed = 3;
const turnspeed = 2;
let lastTime;
```

空的variable（ctx, scene）　空列表[]（voices）　空的（statusBox）

keys: (empty的对象，没有键被按着. no keys are pressing)

enemyAudioBuffers: (空列表，还没加载. empty list, nothing loading yet)

player: (0,-2)，朝前

speed / turnspeed: 3 2 固定不变的数值.

lastTime → 空的. empty. 第一帧才有值, first frame has value.

Step 0-3 → Binding button. 绑定按钮. (on screen ~)

```
document.getElementById('start').addEventListener('click', start);
document.getElementById('end').addEventListener('click', end);
```

(不是start()是因为要点击了才开始，写start() = 马上开始跑，start, = 等点击才跑，先把函数交出去).

Step 0-4 → Starding listning keyboard 开始监听键盘.

```
window.addEventListener('keydown', (e)=>{keys[e.key] = true;});
window.addEventListener('keyup', (e) =>{keys[e.key] = false;});
```

从现在开始，按下任何键都会记到keys里，松开就改回false。只记录，不移动玩家，移动是tick()的事) only record, don't move player

Step 0-5 → denfine All functions

tick. start. loadEnemyBuffers. setStatus, setup.
makeSawVoice, makeFmVoice, makeLoopVoice,
setEnemyVoiceSpeeds, end

写好放那儿，只有被调用时才会运行.

当阶段0结束时. 网页显示出来，按钮能点了，键盘也在被监听，但 没有声音，没有音频工作室，游戏 循环也没开始，一切都在等你点 start

The webpage is displayed, the button can be clicked, the keyboard is being monitored. but → no sound, no audio, and the game loop has not started.

Everything is waiting for you to click Start

Step ①

Step 1-1 → 找到状态框 find status Box.

```
StatusBox = document.getElementById('Status');
```

把网页上id为Status的框存进 statusBox.

用处：状态框就是网页上一行字，用来告诉你游戏里发生了什么. 时实更新游戏发生了什么

就是网页上那一串数字，时实更新.

Step 1-2 → 第一次点：建音频环境 set up()

```
if (ctx == null ){
  set up();
}
```

空的（ctx）

ctx 是空的，进入 setup() function

1. ctx = new AudioContext() 建工作室，必须在用户点击之后才能建.
2. scene = new ResonanceAudio(ctx), build 3D scene, 以后所有声音都放这
3. installAirSorption(...), 越远越闷。
4. installFrontBack(scene, ctx, {cut:15, corner: 300c}), 在背后的高音被削掉.
5. create room: 100x100 x3m. (no walls, like outdoors). and 材料
6. scene.output.connect(ctx.destination) 接到耳机

3. 4. 必须在造任何声源之前装，这样之后造的每个声源都会自动带上效果。

```
let material = "grass";
let roomMaterials = {
  left : 'transparent', right : 'transprant',
  front : 'transparent', back : 'transparent',
  down : material, up : material,
};
let roomDimensions = {width:100, height:3, depth:100};
scene.setRoomProperties(roomDemensions, roomMaterials);
```

! 第二次点 start 时 ctx已经有了→整个跳过，就第一次点的时候建立.
only happen in first time, second time already set up.

Step 1-3. → 加载音频文件 await loadEnemyBuffers()

```
async function loadEnemyBuffers() {
  const soundPaths = ["Sounds/ScaryLoop1.wav", "S.2....wav", "S.3.wav", "Sound/hone.mp3"];
const response = await fetch (Sound Path)
const arrayBuffer = await response.arrayBuffer();
const buffer = await ctx.decodeAudioData(arrayBuffer);
enemyAudioBuffer.push(buffer);
```

soundPaths: 先列出要哪4个文件. (相当于购物清单，用循环一个一个经过下面所有处理)

response: (相当于向服务器要这个文件，拿回来的是一个还未拆开的文件.)

arrayBuffer: (拆开文件. Binary only (只有0.1). 只知道是文件. 但还不知道怎么放声音).

buffer: (翻译 Binary to 能播放的声音，所以这一步必须在setup()建好工作室以后).

enemyAudioBuffer.push: (把翻译好的声音放进 enemyAudioBuffers 这个列表。)

有 await 带表每一步都要take time，"等这步做完在往下走". 这样下一步就不会出错了.

Step 1.4 → 停掉旧声音. Stop voice

```
voices.forEach((voice)=> voice.Stop());
```

对列表里每一个都做一遍（forEach）　拿到一个声音，就叫它停

把 voices 列表里的每个声音，都按下"停止"

为什么要有这一步：第一次点 Start → voice 是空的. 啥都没.
第二次点 Start，如果不停，上次的5个声音还在响，不关掉就变成10个一起响。.

为什么要在 1.5 step 之前. 因为 1-5 会把 voices 换成新列表.
换掉之后，旧声音的遥控器就丢了，再也关不掉. 要趁遥控器在手上，先关。

Step 1·5 → 造5个新声音. build 5 new voices.

```
voice = [
  makeLoopVoice(enemyAudioBuffers[0], [-3.34, -3.74]),
  makeLoopVoice.......     [1], [5.26, -0.78]),
  .......
  .......
  .......];
```

每个voice 都要经一趟"工厂"（"声音制造机"）→ makeLoopVoice() function

```
function makeLoopVoice(buffer, position=[0,0]){
```

buffer: 用哪段录音，从enemyAudioBuffers 里拿

position: 放哪[左右,前后]　[0,0]: 调用时没给位置，default 放在[0,0]

```
let volume =0.5;
```

定音量. (这是正常音量. 0.5) 可调

```
const Source = scene.createSource();
Source.setPosition(position[0]. 0, position[1]);
source.setMaxDistance(25);
source.setGain(volume);
```

createSource: 在3D场景里建一个声源，它负责声音在哪，多大声

setPosition: 放到指定位置 (中间0是高度，地面上)

setMaxDistance: 超过25米听不到.

setGain: 音量设成0.5

建舞台. (声源)

```
const Loopnode = ctx.createBufferSource();
Loopnode.buffer = buffer;
Loopnode.Loop = true;
```

createBufferSource: 建个播放器. 让它发出声音

buffer: 把录音放进去

Loop: 设成无限循环.

建CD机 (播放器).

```
Loopnode.connect(source.input);
```

连线. 录音 → CD 机 → 舞台.

```
Loopnode.Start();
```

开始播放声音!

交出遥控器 (return{...};)

因为source / Loopnode 都是在函数里面建的，外面碰不到.

所以需要交出个 Remote，外面通过按钮来控制它们，没制按钮来一直记住 source, Loopnode. volume，闭包.

```
setPosition(X.Z){
  Source.setPosition(X. 0.Z)
}
```

左右 上下 前后.

```
getPosition(){
    return position
}.
```

not yet.

报告位置，问：在哪. 它回答：[x,y].

```
SetActive(active){
  if (active){ source.setGain(volume);}.
  else       { Source.setGain(0); }
}
```

→ only 静音. 不是停掉.

```
Stop(){
  loopnode.Stop();
}
```

让stop停. 把CD机的(loopnode)关掉. 把声音彻底停止. 相当于关掉CD机

Step 1-6 → 启动游戏循环.

```
requestAnimationFrame(tick);
SetStatus('Playing test tone - should sound like.......');
```

requestAnimationFrame: 让浏览器下一帧开始跑(tick)，让游戏启动！ 按最后因为tick一跑就要用 voices.

SetStatus: 在状态框写一句话. 这是早期留下的旧文字. tick() 第一帧马上用玩家位置把它盖掉. 所以看不到.(基本).

Step 2

tick()

Step 2.1 → 算时间 dt

```
function tick(now){
```

浏览器调用它时自动塞进来的当前时间(毫秒)

游戏心跳开始，顺便告诉你现在几点，拿到时间，才能算这一帧过了多久

```
const t = now / 1000;
```

毫秒 ÷ 1000 = 秒! (相当于把时间单位换算成人用的秒)

```
const dt = lastTime == null ? 0: t - lastTime;
```

算出离上一帧过了几秒，通常为 0.016t

如果第一帧没有上一帧，就当0

用处：移动乘以dt，不管电脑快慢，每秒都走一样远

```
lastTime =t;
```

记住现在几点，下一帧要用它来算dt.

Step 2.2 → 方向键.

```
if (keys["ArrowUp"]){
player.x += speed * dt * Math.cos(player.heading);
player.y ........ * Math.sin.....;
```

上键按着没?

这一帧走多远　朝左右方向上占多少：　前后方向

朝你面对的方向 (cos/sin) 走一小步。实现"坦克式"移动：往哪看就往哪走

上键 ▲

```
if (keys["Arrow Down"]{
  player.x -= speed * .......
  .....
}
```

(一样)个 往后退 ▼

```
if (keys["ArrowLeft"]{
    player.heading += turnspeed * dt;
}
```

向左转. 只改方向. 不改位置!

```
if (keys['Arrowright']{
   ..........(same) -=
}
```

向右转.

Step 2-3 → 数字键.

```
if (key['1']){ voices[0].setActive(true);}.
```

按着"1"键，调用敌人0遥控器的setActive(true) → 音量恢复0.5。开

打开第一个敌人的声音 (测试用)

```
.......[2] ......(false)
```

→ 表示静音 关

以此类推 → "3"打开 "4"关上

Step 2.4 → 显示玩家位置.

```
setStatus('player position ${JSON.Stringify(player)});
```

写进状态框.

把对象变成文字{"x":0,"y"-2,"heading":1.57}; ${...}把它塞进句子

${...}把它塞进句子.

在网页上显示你现在在哪，朝哪.

调试

Step 2.5 → 告诉Resonance你的耳朵.

```
Scene.setListenerPosition(player.x, 0, player.y);
```

Resonance 用(x,高度,前后)

告诉3D场景"听的人现在站在这里."

用处：你移动了，声音的远近和方向要跟着变.

必须在 2.2 移动之后.

```
scene.setListenerOrientation(Math.cos(player.heading), 0, Math.sin(player.heading), 0, 1, 0);
```

告诉 3D 场景："听到的人脸朝这边。"

前方　鼻子方向　　头顶方　上方.

Step 2-6 → //掉的 code.

```
//voices[0].setPosition(Math.cos(Math.PI*t) * 2, Math.sin(Math.PI*t) * 2);
// voices[1].setPosition(-Math.cos(Math.PI*t /2)*2 ,-Math.sin(Math.PI*t/2)*2);
//setStatus(dt);
```

~~让两个敌人绕圈走~~

显示 dt 的测试代码

Step 2-7 → 预约下一帧

```
request AnimationFrame(tick);
}.
```

这一帧做完了，下一帧再来一遍.

形成循环，游戏一直跑下去.

every frame: 算时间 → 看方向键移动转向 → 看数字开关声音 → 显示位置 → 告诉Resonance 耳朵在哪，朝哪 → 预约下一帧

Step 3

end()

```
function end(){
    voice.forEach((voice) => voice.stop());
}
```

点 End 对每个声音按 Stop

CD 机全关 停掉

! tick() 还在跑，还能走，只是听不到 .; 再点 start会回到 Step -1 . set up() 跳过

! 同一行 code 和 1-4，但 1-4 是再点start 时自动关旧的。
但end()是你手动点 End，按键.